

MLOps Lecture: Kubernetes Fundamentals, Concepts, Best Practices, and Pitfalls

AI 545 / MLOps

1 Introduction

Modern machine learning systems require reliable deployment, reproducibility, scalability, and fault tolerance. Unlike small experimental projects, production machine learning systems must continuously run, serve predictions, handle multiple users, and recover from failures automatically. Managing these requirements manually becomes impossible when the system grows beyond a single machine. For this reason, modern ML systems rely on containerized applications and orchestration platforms.

Kubernetes is the industry standard container orchestration system used to automate deployment, scaling, and management of containerized applications. In the context of MLOps, Kubernetes is commonly used to run training jobs, deploy model inference services, manage pipelines, and maintain high availability of machine learning systems.

In simple terms, Docker allows us to package an application into a container, but Kubernetes allows us to manage hundreds or thousands of containers across multiple machines in a reliable and automated way. This lecture introduces the core concepts of Kubernetes, its architecture, best practices, and common mistakes, with emphasis on how these ideas relate to machine learning systems.

2 Why Kubernetes is Needed in MLOps

A production machine learning system is not only a trained model. A real system may include a training pipeline, data preprocessing scripts, model registry, inference API, database, monitoring service, logging system, and batch processing jobs. Each of these components may run on different machines and may need to scale independently. For example, the training job may require a GPU, while the inference API may require multiple CPU replicas to handle user requests.

If such a system is managed manually, several problems arise. When a container crashes, it must be restarted manually. When traffic increases, new instances must be started manually. When a server fails, the entire service may become unavailable. Updates are also difficult because replacing a running service without downtime requires careful coordination.

Docker solves the problem of packaging software, but it does not solve the problem of orchestration. Orchestration means deciding where containers run, how many copies exist, when to restart them, and how to connect them to each other. Kubernetes provides this orchestration layer and makes it possible to operate complex machine learning systems reliably.

3 Containers and Orchestration

A container is an isolated environment that contains an application together with its dependencies. In machine learning, this usually includes Python, required libraries, trained model files, and configuration files. By using containers, we ensure that the application runs the same way on a laptop, on a server, or in the cloud.

However, when multiple containers must run together, additional management is required. Suppose an ML inference service must run five copies of the same container to handle incoming requests. If one container crashes, a new one should start automatically. If traffic increases, more copies should be created. If a machine fails, containers should move to another machine. These tasks cannot be handled by Docker alone. Kubernetes provides a scheduler, controllers, and networking components that automate these tasks and allow the system to operate without manual intervention.

4 Kubernetes Cluster Architecture

Kubernetes operates as a cluster of machines. A cluster consists of a control plane and one or more worker nodes. The control plane is responsible for making decisions about the cluster, while worker nodes run the actual containers. This separation allows Kubernetes to manage large systems reliably because the control logic is independent from the machines that execute the workloads.

The control plane includes several components. The API server is the central entry point that receives all commands. The scheduler decides where new containers should run based on available resources. The controller manager ensures that the actual state of the system matches the desired state. The etcd database stores the cluster configuration and state information. These components together ensure that the system continuously moves toward the desired configuration.

Worker nodes are the machines that run the containers. Each worker node runs a program called kubelet that communicates with the control plane and ensures that the required containers are running. The node also includes a container runtime, such as containerd, which actually executes the containers. A networking component called kube-proxy ensures that containers can communicate with each other across different machines.

In MLOps, this architecture allows training jobs, inference services, and data processing tasks to run on different nodes while still being managed as a single system.

5 Pods

The smallest deployable unit in Kubernetes is called a Pod. A pod represents one or more containers that run together on the same machine and share the same network and storage. In most machine learning applications, a pod contains a single container, such as a model server or a training script. However, multiple containers may be placed in the same pod if they must run closely together, for example a model server and a logging sidecar container.

Pods are designed to be temporary. If a pod fails, Kubernetes does not repair the same pod but instead creates a new one. This design makes the system more reliable because the state is not stored inside the pod but outside it. In MLOps, this means that model files, configuration, and data should not be stored inside the container itself but in external storage or configuration objects.

6 Deployments

A Deployment is a higher-level object that manages pods automatically. Instead of creating pods directly, we define a deployment that describes how many copies should exist and which container image should be used. Kubernetes then creates and maintains the required pods.

Deployments are essential for production systems because they allow automatic restart, scaling, and updates. For example, suppose a machine learning inference service must always have three running copies. If one pod crashes, the deployment automatically creates a replacement. If a new version of the model must be deployed, the deployment can gradually replace old pods with new ones without stopping the service.

This feature is extremely important in MLOps because models are updated frequently, and downtime during deployment is usually unacceptable.

7 ReplicaSets

ReplicaSets are responsible for ensuring that a specified number of pods always exist. A deployment internally uses ReplicaSets to maintain the desired number of replicas. If the number of running pods becomes smaller than expected, new pods are created automatically. If too many pods exist, extra ones are removed.

In a machine learning system, this mechanism is useful for inference services. When many users send requests to a model, multiple replicas can handle the load. If one replica fails, another is created automatically, ensuring that the service remains available.

8 Services

Pods are temporary and their network addresses may change. Because of this, other components cannot connect directly to a pod. Kubernetes provides a Service object that gives a stable address and automatically forwards traffic to the correct pods.

For example, an API server may need to send requests to a model server. Instead of connecting to a specific pod, the API connects to the service. The service then distributes requests among available pods. This mechanism provides load balancing and makes the system robust to failures.

In MLOps, services are used to expose model inference endpoints, connect microservices, and provide stable access to databases and feature stores.

9 Scaling

One of the main advantages of Kubernetes is the ability to scale applications automatically. Scaling means increasing or decreasing the number of running pods based on resource usage or user traffic. Horizontal scaling creates more copies of a pod, while vertical scaling increases the resources assigned to a pod.

In a machine learning inference system, scaling is necessary because the number of user requests may change over time. During peak hours, more replicas may be needed, while at night fewer replicas are sufficient. Kubernetes can monitor CPU usage or custom metrics and adjust the number of pods automatically. This feature reduces cost and improves performance.

10 Self-Healing

Kubernetes continuously monitors the state of the system. If a container crashes, Kubernetes restarts it. If a pod fails, a new pod is created. If a node becomes unavailable, pods are scheduled on another node. This behavior is called self-healing and is one of the main reasons Kubernetes is used in production systems.

In machine learning applications, self-healing ensures that inference services remain available even when hardware fails or software crashes. Without this feature, manual intervention would be required to restore the system.

11 Rolling Updates

Updating a running system without downtime is difficult. Kubernetes provides rolling updates, which gradually replace old pods with new ones. At any time, some old pods remain running while new pods start, ensuring that the service continues to operate.

This feature is particularly important in MLOps because new model versions must be deployed frequently. Rolling updates allow safe deployment of new models and make it possible to roll back to the previous version if a problem occurs.

12 Configuration Management

Applications should not store configuration values inside the container image. Instead, Kubernetes provides ConfigMaps and Secrets to store configuration data separately from the application. ConfigMaps are used for general configuration, while Secrets are used for sensitive information such as passwords or API keys.

In machine learning systems, configuration may include database addresses, model paths, or external service keys. Separating configuration from the container makes the system more flexible and secure.

13 Best Practices

Reliable Kubernetes systems require careful design. Containers should be small and contain only the required software. Images should be versioned so that deployments are reproducible. Resource limits should be defined to prevent one container from consuming all available memory or CPU. Health checks should be used so that Kubernetes can detect failed containers. Rolling updates should always be used instead of stopping and restarting services. Configuration should never be hard-coded inside the image.

In MLOps, additional best practices include separating training and inference workloads, versioning models, using external storage for datasets, and monitoring system performance. Following these practices makes the system easier to maintain and reduces the risk of failure.

14 Common Mistakes

Beginners often create large containers that include many unrelated components. This makes the system difficult to update and debug. Another common mistake is running containers without resource limits, which can cause the entire node to become unstable. Hard-coded configuration

values make deployments inflexible. Lack of logging and monitoring makes debugging very difficult. Running stateful applications inside pods without external storage can lead to data loss.

In machine learning systems, a frequent mistake is storing model files inside the container image. This makes updates slow and prevents efficient versioning. Another mistake is running training and inference in the same container, which causes resource conflicts.

15 Pitfalls in MLOps

Machine learning workloads have additional challenges compared to normal web applications. Training jobs may run for hours or days and require GPUs. Models may be very large and require special storage. Data pipelines may depend on external datasets. These requirements mean that Kubernetes must be configured carefully.

For example, GPU scheduling must be enabled to allow training jobs to use hardware accelerators. Jobs should be used instead of deployments for batch training tasks. Persistent volumes should be used for datasets and model storage. Without proper configuration, the system may become slow, unreliable, or expensive.

16 Conclusion

Kubernetes is a container orchestration platform that automates deployment, scaling, and management of applications. In MLOps, it provides the infrastructure required to run reliable machine learning systems. By understanding pods, deployments, services, scaling, and configuration management, it becomes possible to build production-grade ML systems that can operate continuously without manual intervention.

17 References

1. Kubernetes Documentation — What is Kubernetes <https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/>
2. Kubernetes Architecture <https://kubernetes.io/docs/concepts/architecture/>
3. Pods <https://kubernetes.io/docs/concepts/workloads/pods/>
4. Deployments <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
5. ReplicaSets <https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>
6. Services <https://kubernetes.io/docs/concepts/services-networking/service/>
7. Autoscaling <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
8. Configuration <https://kubernetes.io/docs/concepts/configuration/>
9. Jobs <https://kubernetes.io/docs/concepts/workloads/controllers/job/>
10. Kubernetes Best Practices <https://kubernetes.io/docs/concepts/configuration/overview/>